

RRSS

Architecture Notebook

1. Purpose

The purpose of this document is to provide a comprehensive architectural overview of the RRSS. It outlines the fundamental principles, and overall considerations that shape the development and implementation of the system. It serves as a complete guide for stakeholders, developers and architects to understand the core concepts and decisions that drive the architecture.

2. Architectural goals and philosophy

ShopSmart aims to provide a seamless platform that connects customers and sellers, facilitating efficient product discovery, transactions and community engagement. The architectural philosophy focuses on creating an intuitive, reliable and scalable system that enhances the user experience while ensuring long-term maintainability and extensibility.

- **User-centric design:** The architecture prioritizes the user experience, aiming for simplicity, ease of use and accessibility across devices. Customers and sellers should find it effortless to navigate, search for products and interact with each other.
- **Scalability and performance:** As the platform grows, the architecture should be able to scale horizontally and vertically to accommodate increasing user traffic and data volumes. Performance remains a key consideration. Ensuring fast response times and smooth operation even during peak loads.
- **Reliability and Availability:** Platform should be highly available with minimal downtime. The architecture must provide robust fault tolerance, redundancy and recovery mechanisms to ensure uninterrupted service.
- **Security and Privacy:** The protection of user data and transactions is critical. The architecture must implement strong security measures, including encryption, authentication and authorisation, to safeguard sensitive information and maintain user confidence.
- **Flexibility and Extensibility:** The architecture should support easy integration with third-party services and APIs, allowing for future expansion and incorporation of new features. Modular components and well-defined interfaces facilitate adaptability to changing business needs.
- **Simplicity:** Aim for simplicity in design and implementation without sacrificing functionality. Complexity should be hidden behind intuitive interfaces, making the platform easy to understand and use.

3. Assumptions and dependencies

Assumptions that drive the architectural decisions are listed below:

- The website will be accessed through modern web browsers on mobile and desktop class devices.
- Users have uninterrupted internet connection while using the website.
- Spring framework for Java will be sufficient for handling backend tasks.
- SQLite database will be sufficient for handling the data load and complexity considering the scale of the project.
- The team has necessary knowledge to work with the tech stack chosen for this project.

Dependencies for the project are listed below:

- The proper look of the frontend part of the website is reliant on the compatibility of HTML, CSS and Bootstrap with different browsers and devices.
- The website's performance is dependent on the efficiency of the SQLite database under concurrent access.

- The reliability and stability of the website is dependent on the chosen hosting service and deployment method.

4. Architecturally significant requirements

Architecturally Significant Requirements (ASRs) are those requirements that have a measurable effect on a computer system's architecture. They determine the structure and foundational decisions in system design. And they are often non-negotiable, constraining, and strict.

The ASRs we identified for this project are listed below:

- Communication between the user and the server is done through the HTML methods (GET, POST etc.)
https://en.wikipedia.org/wiki/HTTP#Request_methods
- Passwords that are stored in the database are hashed using the Argon2 algorithm.
<https://en.wikipedia.org/wiki/Argon2>
- User sessions are handled using Spring Security framework.
<https://spring.io/projects/spring-security>
- Mistral 7B is the Large Language Model used for the ChatBot feature.
<https://mistral.ai/news/announcing-mistral-7b/>
- Bootstrap library will be used for styling the website.
[https://en.wikipedia.org/wiki/Bootstrap_\(front-end_framework\)](https://en.wikipedia.org/wiki/Bootstrap_(front-end_framework))

5. Decisions, constraints, and justifications

▪ Monolithic Architecture with Standalone ChatBot

Develop the core system as a single application, with a separate, deployable ChatBot service. This will simplify the initial development and deployment of the project. As it won't be a scaling-intensive project, using Monolithic Architecture will not degrade the performance ideally. Using standalone ChatBot service will isolate complexity and allows for independent development and testing of the ChatBot functionality.

▪ SQL Database

Utilize a relational database, SQLite, for structured data. This decision will provide well-suited structure and querying capabilities for the data we are going to use. Moreover, it will make the connection between our system design, described in UML diagrams, with the codebase more suited to each other.

▪ Use Thymeleaf Template Engine

Using Thymeleaf Template Engine will make the development much easier. Considering the timeline of the project, and the developers' workload, this decision sounds quite sane compared to frameworks like React which has a very steep learning curve. Moreover, one of the main concerns of our system is to keep things as simple as possible due to limited resources. Therefore, fancy user experience and interfaces can be sacrificed without thinking twice.

▪ Recommendation Algorithm Complexity

With the help of browsing history and other relevant data, recommendation algorithm will be implemented and ideally work as expected. Implementation will be based on scoring. Therefore, a second Artificial Intelligence system won't be needed.

▪ Separation of Presentation, Logic, and Data

Design the system with a layered architecture, separating presentation, business logic, and data access layers. A layered architecture promotes loose coupling and maintainability, allowing for easier future integration of new features or services without major code rewrites.

▪ Mistral 7B as Large Language Model (LLM)

Utilize Mistral 7B as the LLM for the ChatBot functionality within the RRSS. Mistral 7B offers a lightweight, powerful LLM for the ChatBot, ideal for our monolithic architecture. Additionally, its free access aligns well with our project constraints.

6. Architectural Mechanisms

- **Authentication Mechanism**

The Authentication Mechanism ensures secure user login and access control within the RRSS. It verifies user credentials to grant access. This mechanism manages user sessions, ensuring only authorized users can access specific functionalities based on their assigned access levels. It will involve user accounts with password hashing for security, and a session management utility to track user's login status.

During login, the authentication mechanism verifies user credentials against the user database. Upon successful login, it creates a user session. This session allows the user to interact with the system until they log out, at which point the session is terminated.

- **Data Persistence Mechanism**

The Data Persistence Mechanism provides a reliable way to store and manage various types of data within the RRSS. The core attribute is the chosen database type. A relational database is suitable for its structured query capabilities. Additionally, a data access layer will be needed to interact with the database using standardized queries regardless of the underlying database technology.

The Data Persistence Mechanism serves as the central storage for your system's data. It allows different parts of the system (e.g., product search, recommendation engine) to retrieve and manipulate data as needed. The data access layer acts as an intermediary, translating business logic into database operations and ensuring data integrity.

- **Messaging Mechanism**

The Messaging System facilitates asynchronous communication between the ChatBot service and other parts of the RRMS. This allows the ChatBot to send requests for information and receive responses independently, without blocking other functionalities within the monolithic application. The core attribute is the chosen protocol and it is going to be HTTP.

RRSS system sends messages containing user queries and earlier chats to the ChatBot system. Once it's processed, ChatBot service responds to the user messages. This asynchronous communication allows the RRSS to remain responsive to user queries even while waiting for responses from other services.

7. Key abstractions

- **Web Framework:** The Spring Boot Framework will be used to abstractize away the HTTP code, enabling us to focus on developing the business layer logic using the controller architecture.
- **Object-Relational Mapping:** Spring Data JPA will be used in order to abstractize away the model layer operations. The abstraction will be especially useful if the current database is deemed insufficient in the future.
- **Deployment System:** The docker-compose system will be used to abstractize away the deployment details. The system will ensure that the system will run without hiccups in systems with different configurations.

8. Layers or architectural framework

The RRSS application will leverage a layered architecture combined with a separate service for the Chatbot functionality. This approach offers several advantages:

- **Separation of Concerns:** The layered architecture promotes a clean separation of concerns by dividing the application into distinct layers: presentation (user interface), business logic (core functionalities), and data access (database interaction). This modularity simplifies development, testing, and maintenance. Each layer

can be independently developed, updated, and scaled without impacting the others.

- **Scalability and Performance:** The Chatbot, as a separate service, can be independently scaled to meet its specific processing demands. If the Chatbot requires significant computational resources due to its large language model integration, its architecture allows for horizontal scaling by adding more instances of the service. This ensures optimal performance for both the Chatbot and the core RRSS application.
- **Maintainability and Reusability:** The layered architecture promotes maintainability by compartmentalizing functionalities. Changes within a layer are less likely to impact other layers. Additionally, the Chatbot service can be potentially reused in other applications if needed, enhancing its overall value.
- **Reduced Complexity:** By isolating the Chatbot functionality in a separate microservice, the core RRSS application remains simpler and easier to manage. This reduces the overall complexity of the system, making it more manageable for development and deployment teams.
- **Inter Service Communication:** The chosen messaging mechanism (described in section 6) will facilitate communication between the Chatbot microservice and the core RRSS application. This asynchronous communication allows the main application to remain responsive to user queries while the Chatbot processes information and generates responses.

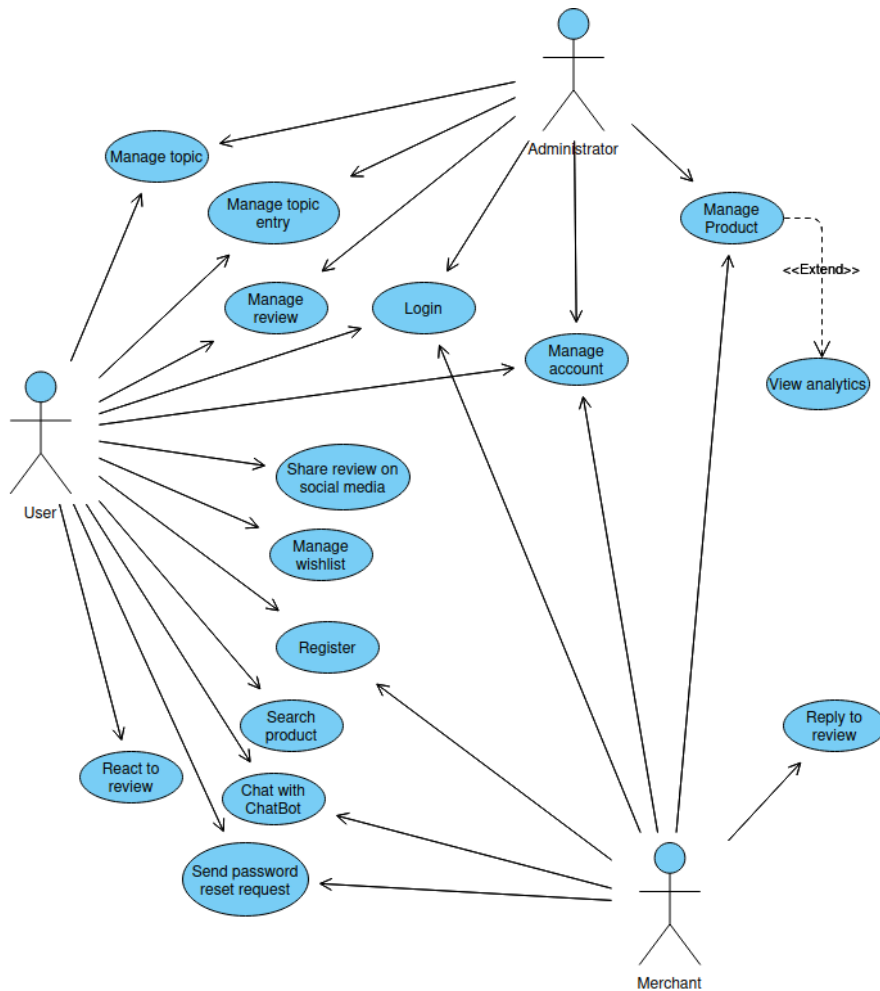
9. Architectural views (İlker)

Architectural views provide different perspectives on the system's architecture, offering a comprehensive understanding for stakeholders. Here's a breakdown of the recommended views for the RRSS project:

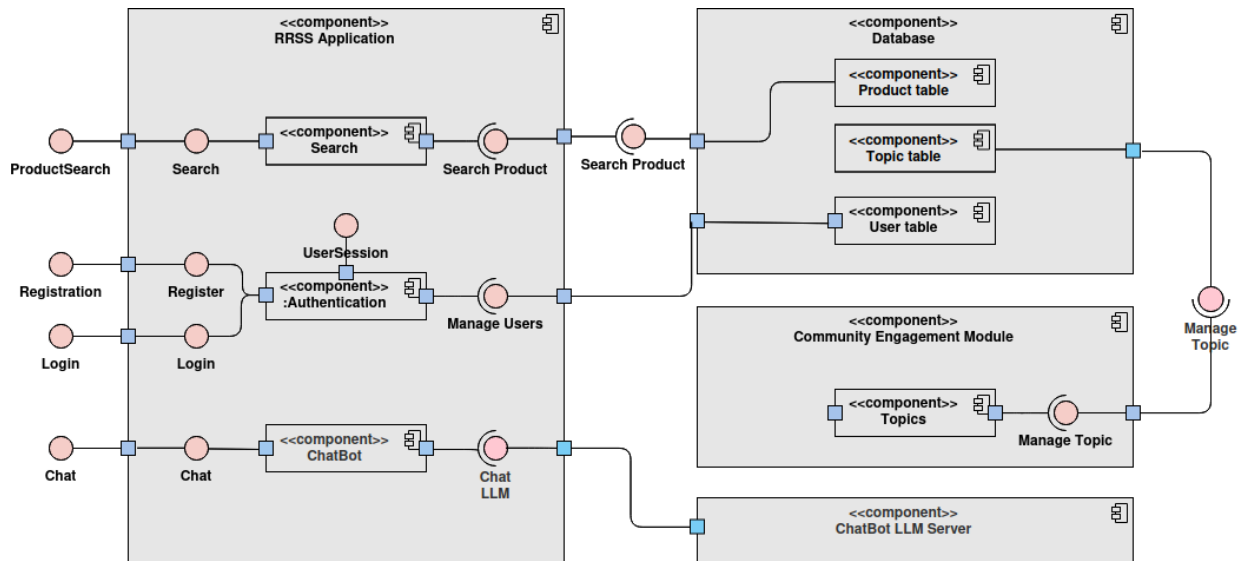
- **Logical View:** This view focuses on the high-level structure and behavior of the system. It should depict the following key elements:
 - **Layered Architecture:** Visually represent the separation into presentation, business logic, and data access layers.
 - **Chatbot Service:** Illustrate the Chatbot service as a distinct component interacting with the core RRSS application.
 - **Data Access Layer:** Show the data access layer interacting with the chosen database (SQLite in this case).
 - **Component Interactions:** Clearly define the interfaces and communication pathways between different components (e.g., how the presentation layer interacts with the business logic layer, how the Chatbot service retrieves data from the core application).
- **Use Case View:** This view focuses on the system's functionalities from the user's perspective. A Use Case Diagram is a good way to represent this view. It should illustrate how different user roles (seller, customer, etc.) interact with the system to achieve specific goals. For example, the diagram could show how a customer searches for products, interacts with the Chatbot for recommendations, and completes a purchase.

Appendices

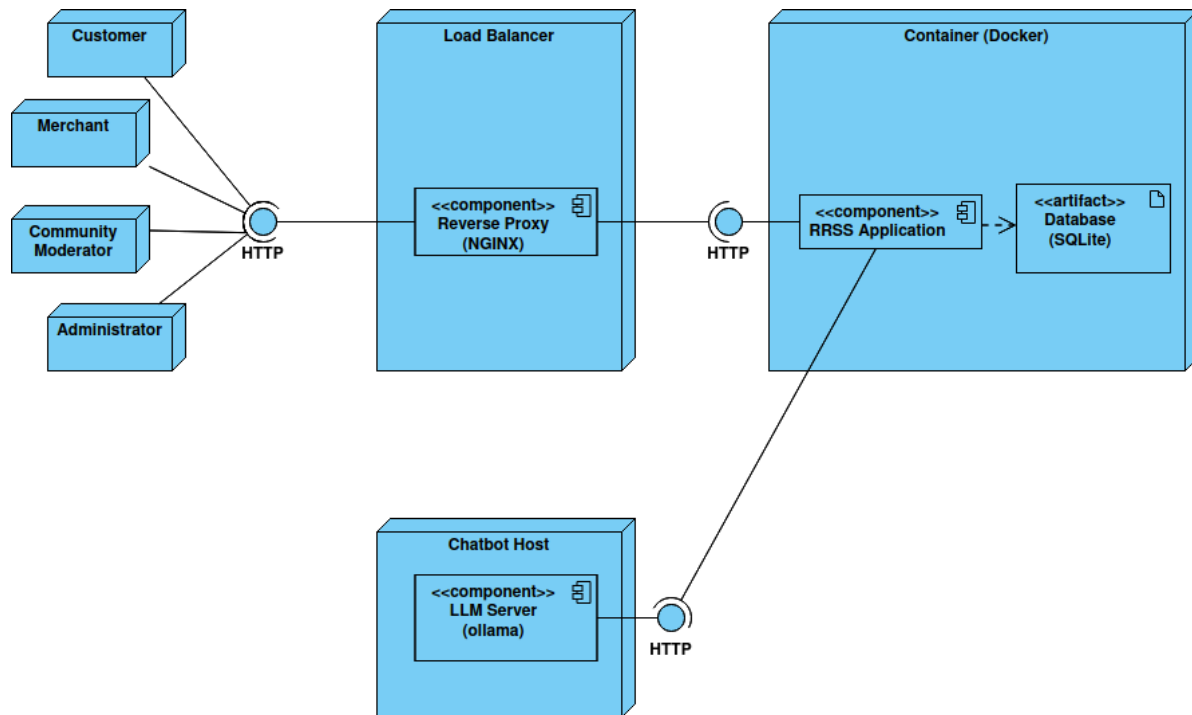
Updated Use Case Diagram:

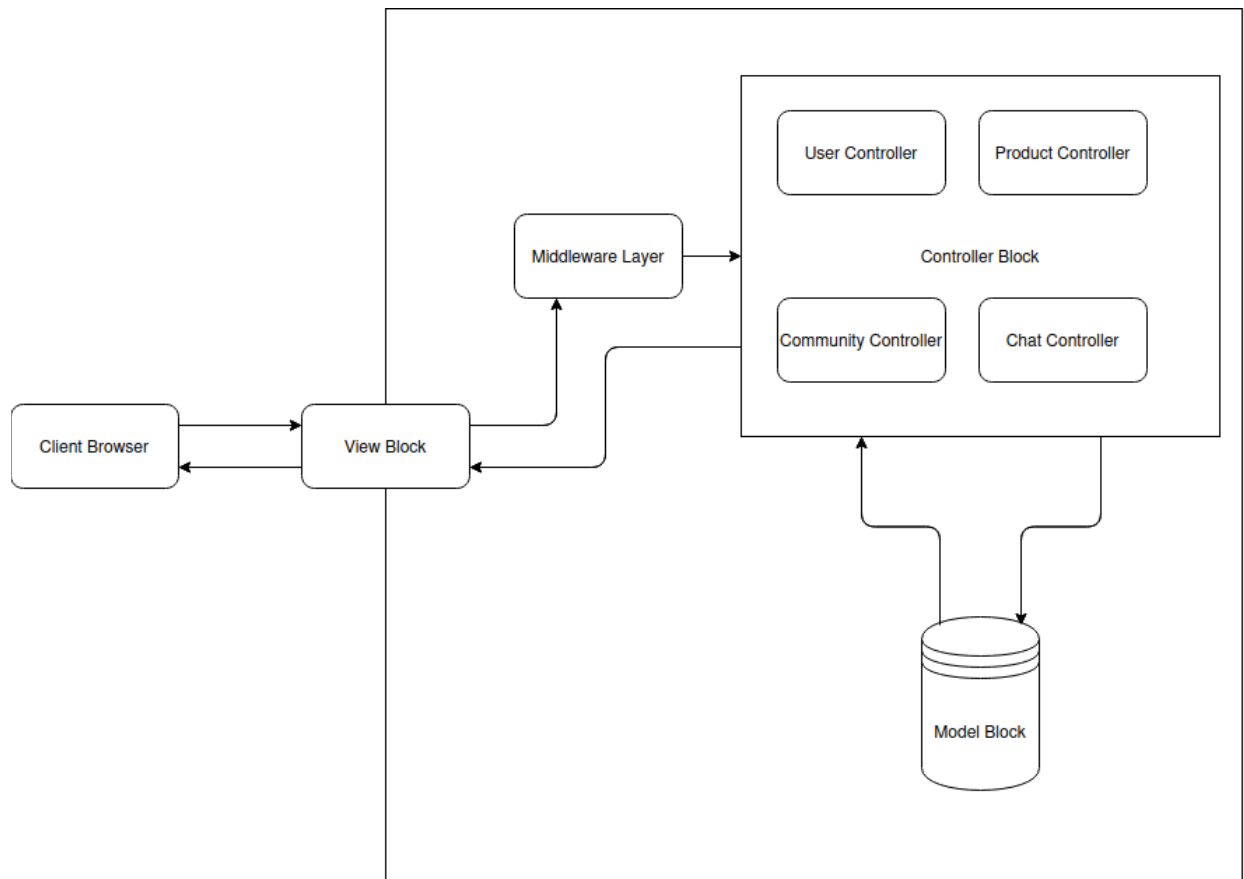


Component Diagram:

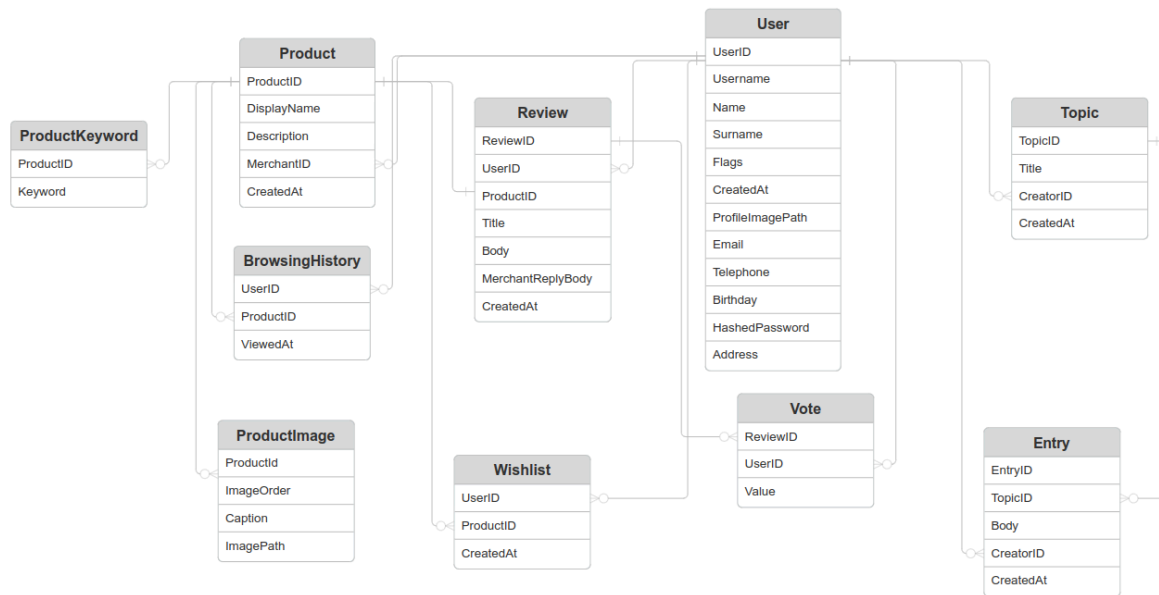


Deployment Diagram:

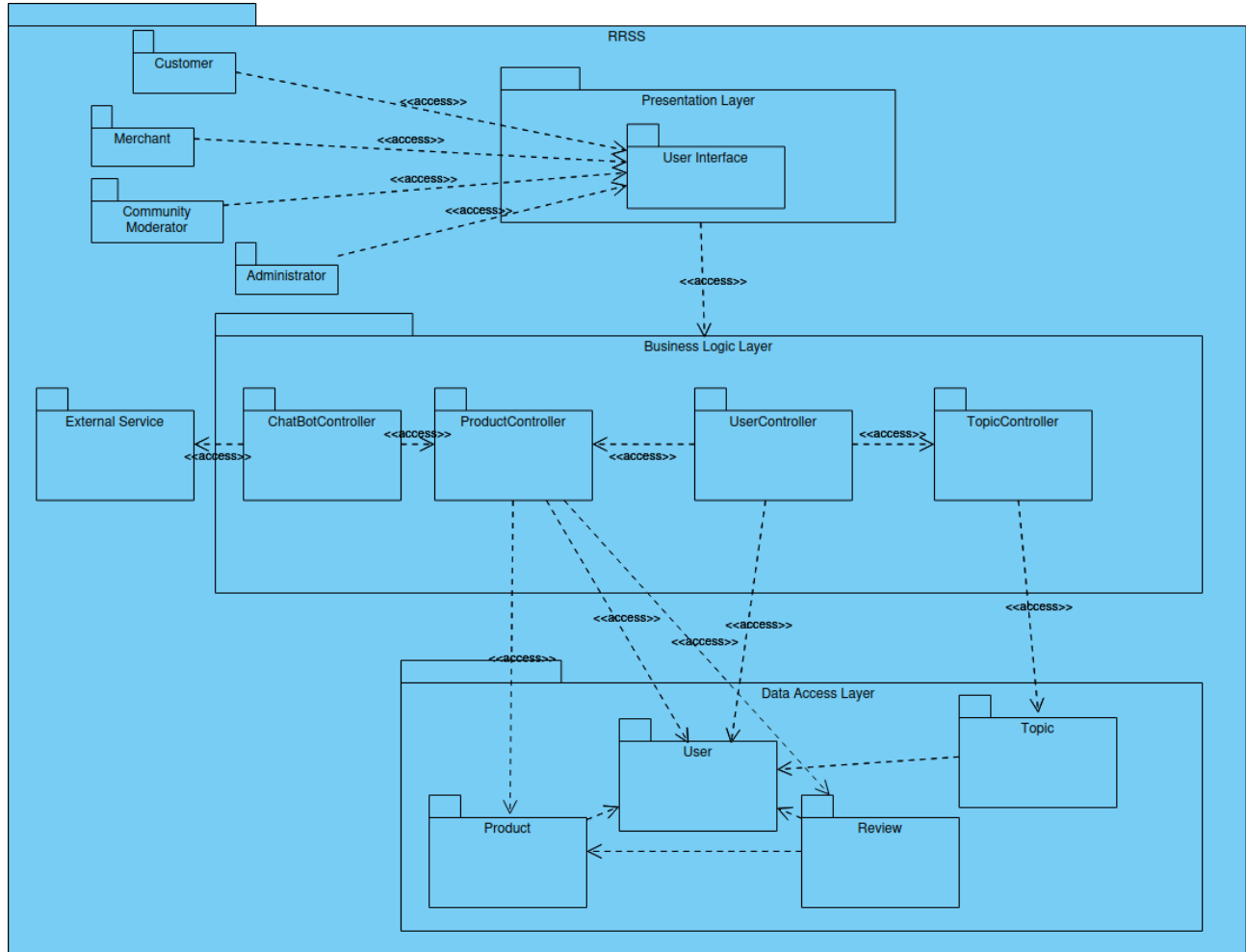


Block Diagram:

Updated ER Diagram:



Package Diagram:



Class Diagram:

